

Lecture 6:

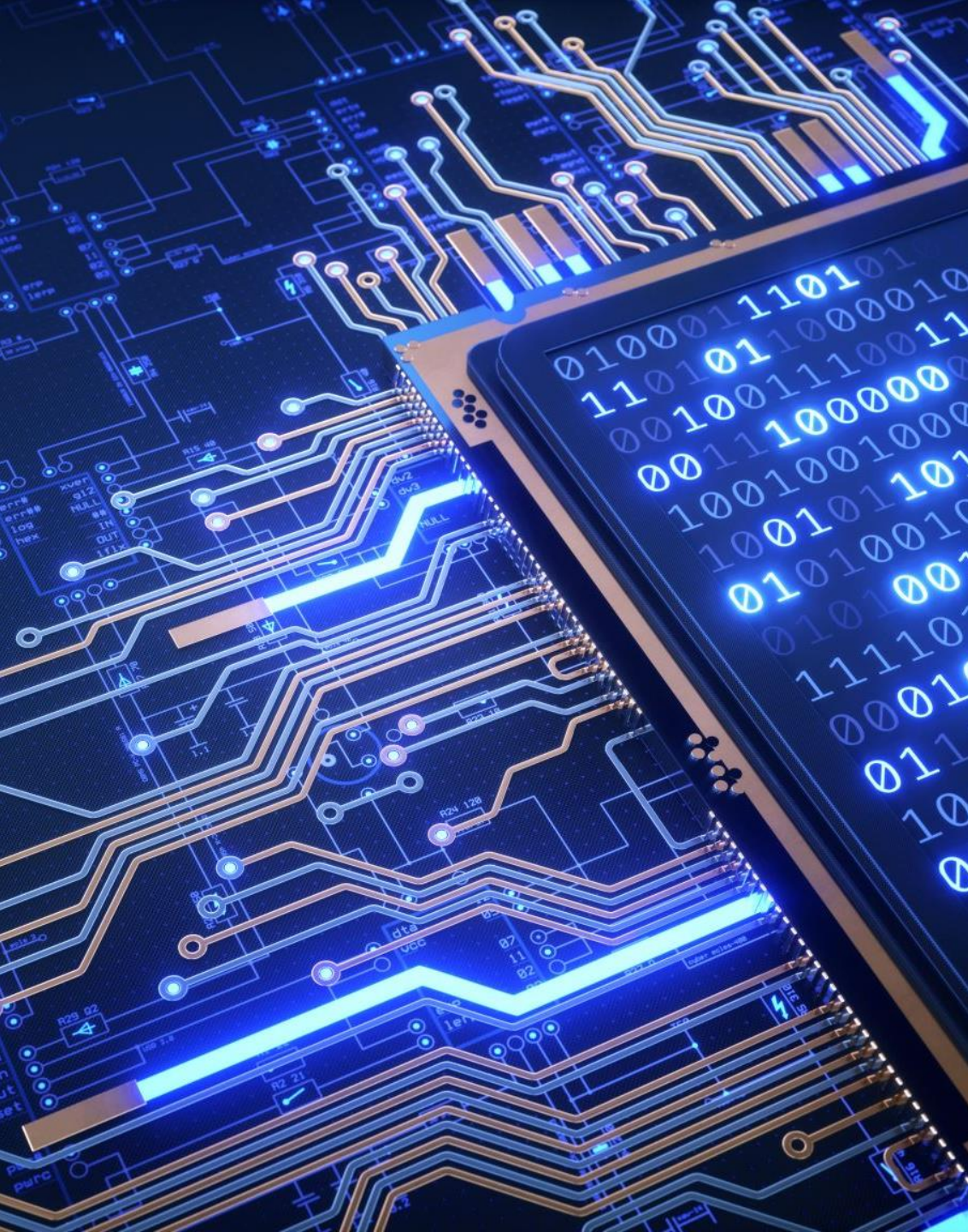
Application Layer

Protocol Ep.2 : Mail (SMTP), Streaming, CDNs

R wiangsripanawan

30.07.24





Outcome : Mail

- Understand how mail messages are travelled from a sender to a receiver
- Understand SMTP, IMAP and POP3

Outline

Mail: Components

- **User Agents**
- **Mail Servers**
 - **Outgoing Msg queue**
 - **User mailbox**

Mail Protocols

- **Main : SMTP**
 - **UA->Sender Mail Server**
 - **Sender Mail Server -> Receiver Mail Server**
- **From Rcv Mail server=> Client**
 - **IMAP**
 - **POP3**

Mail



Slide page 55 – 62: Email, SMTP, IMAP



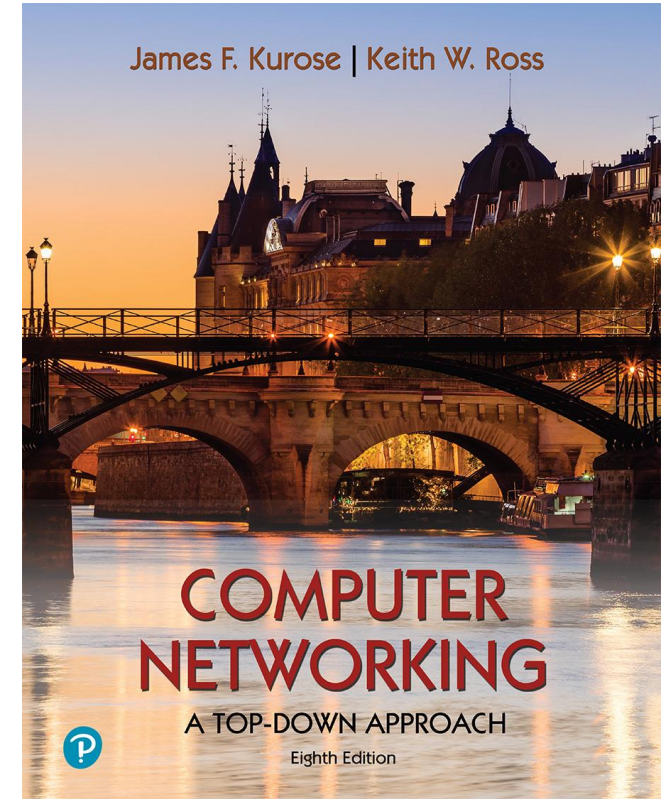
นำมาจาก **Powerpoint slides**
Chapter 2: Application Layer

Page 2-55 – 2-62: Email, SMTP,
IMAP



**Book : Computer Networking: A
Top-Down Approach 8th edition**
โดย **Jim Kurose, Keith Ross,**
Pearson, 2020

https://gaia.cs.umass.edu/kurose_ross/ppt.php



*Computer Networking: A
Top-Down Approach*

8th edition

Jim Kurose, Keith Ross
Pearson, 2020

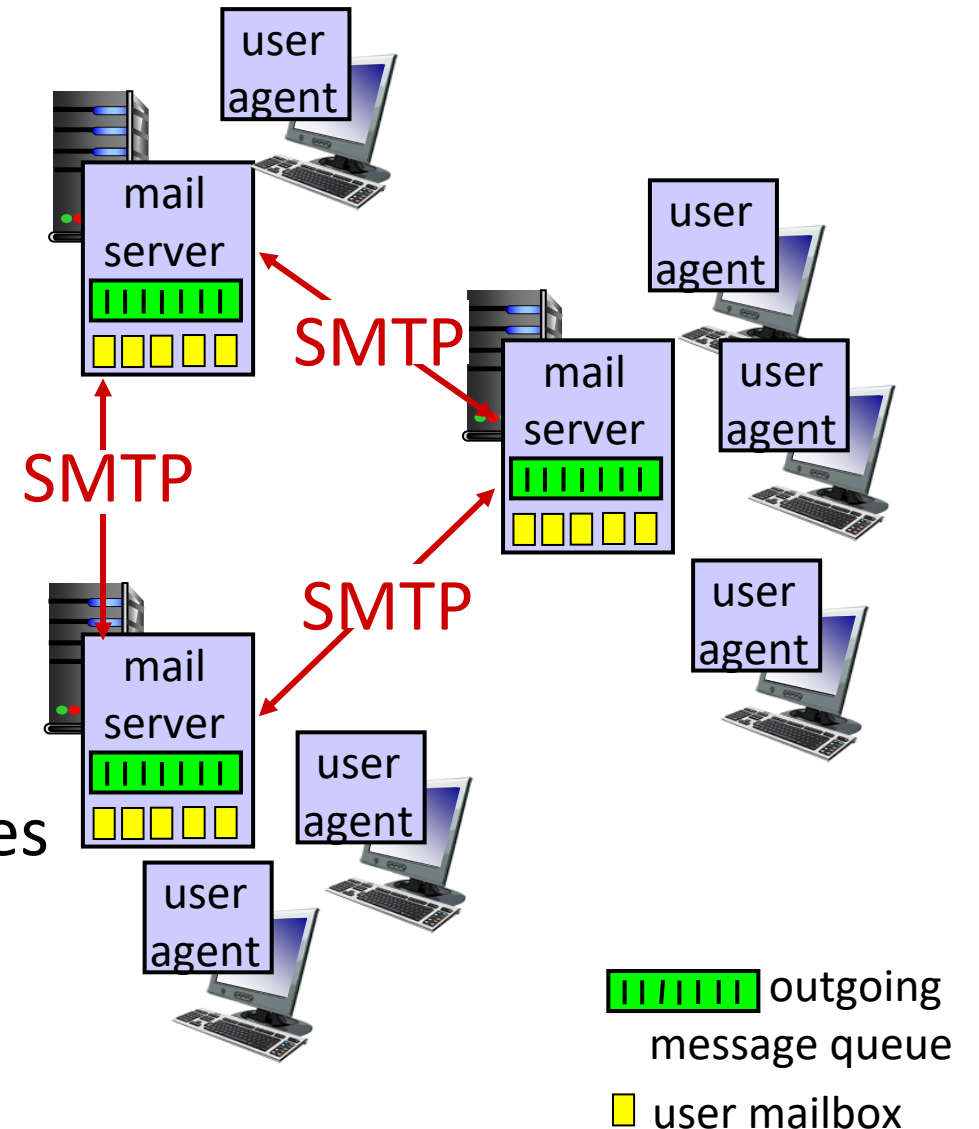
E-mail

Three major components:

- user agents
- mail servers
- simple mail transfer protocol: SMTP

User Agent

- a.k.a. “mail reader”
- composing, editing, reading mail messages
- e.g., Outlook, iPhone mail client
- outgoing, incoming messages stored on server



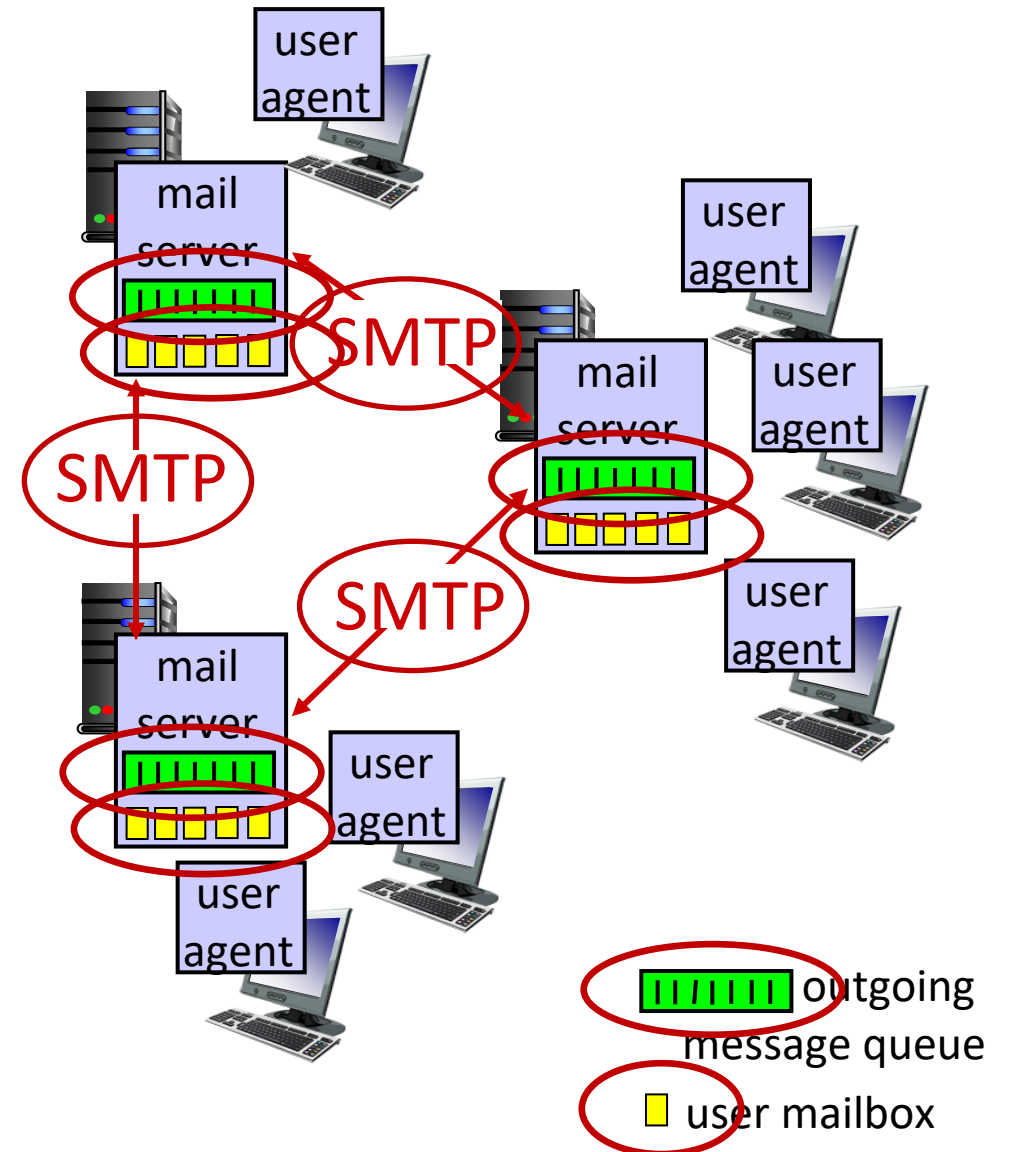
E-mail: mail servers

mail servers:

- *mailbox* contains incoming messages for user
- *message queue* of outgoing (to be sent) mail messages

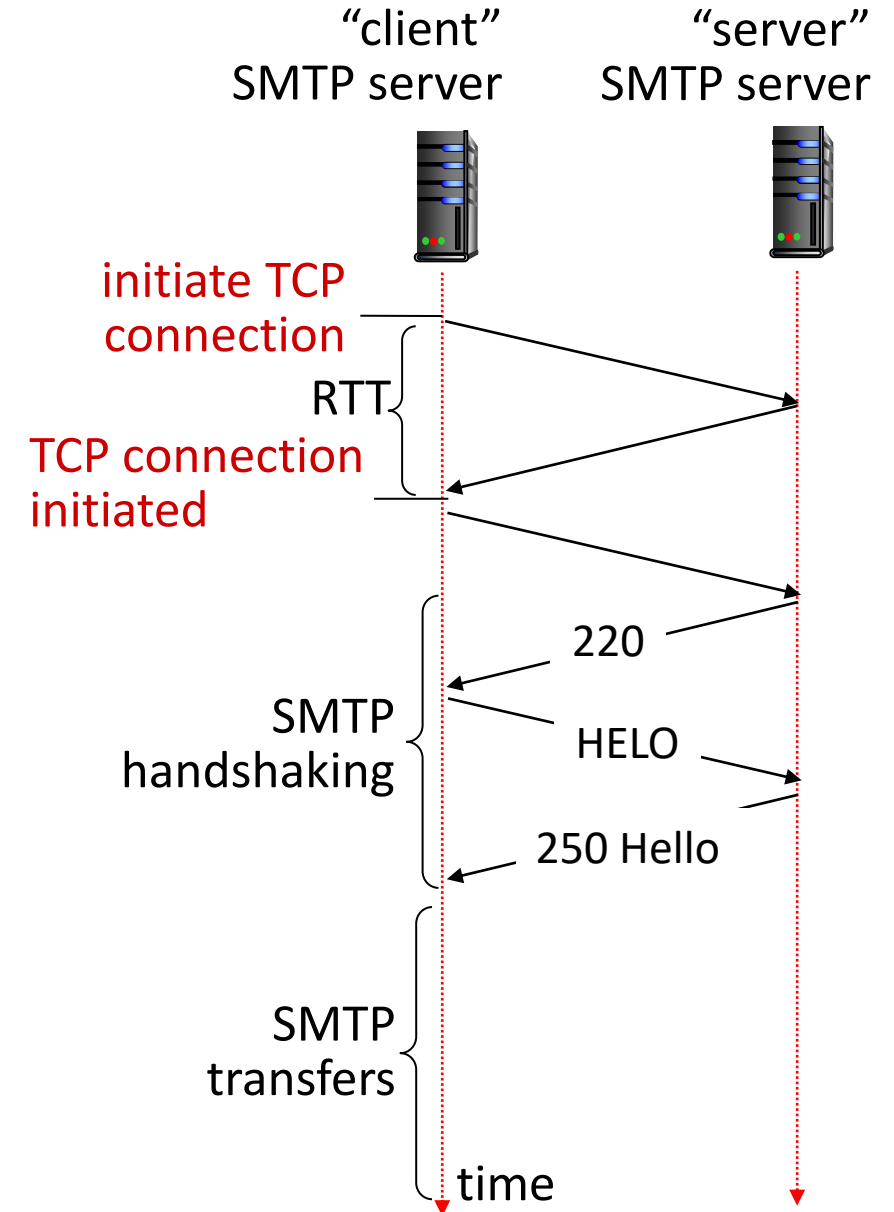
SMTP protocol between mail servers to send email messages

- *client*: sending mail server
- “*server*”: receiving mail server



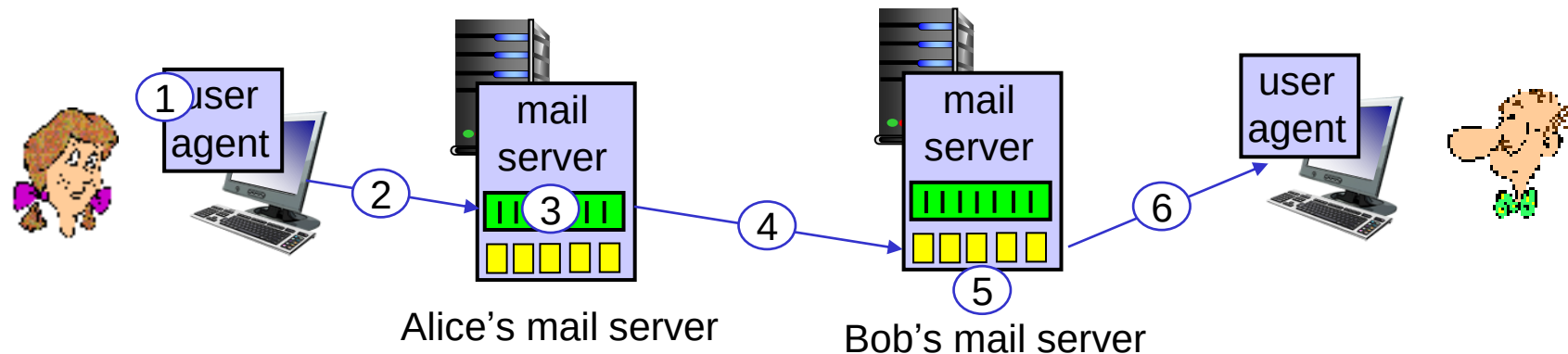
SMTP RFC (5321)

- uses TCP to reliably transfer email message from client (mail server initiating connection) to server, port 25
 - direct transfer: sending server (acting like client) to receiving server
- three phases of transfer
 - SMTP handshaking (greeting)
 - SMTP transfer of messages
 - SMTP closure
- command/response interaction (like HTTP)
 - **commands**: ASCII text
 - **response**: status code and phrase



Scenario: Alice sends e-mail to Bob

- 1) Alice uses UA to compose e-mail message "to" bob@someschool.edu
- 2) Alice's UA sends message to her mail server using SMTP; message placed in message queue
- 3) client side of SMTP at mail server opens TCP connection with Bob's mail server
- 4) SMTP client sends Alice's message over the TCP connection
- 5) Bob's mail server places the message in Bob's mailbox
- 6) Bob invokes his user agent to read message



Sample SMTP interaction

S: 220 hamburger.edu

SMTP: observations

comparison with HTTP:

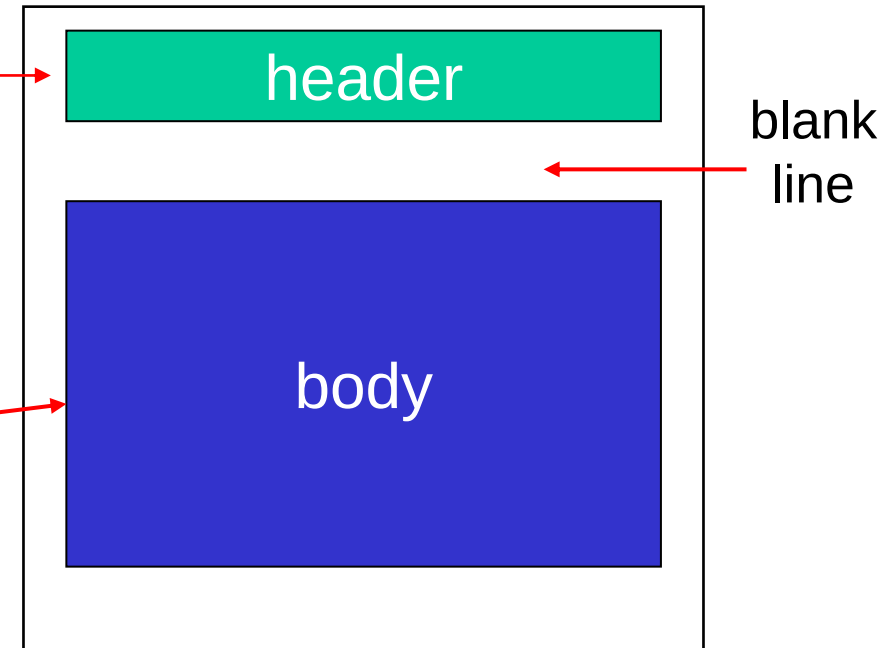
- HTTP: client pull
 - SMTP: client push
 - both have ASCII command/response interaction, status codes
 - HTTP: each object encapsulated in its own response message
 - SMTP: multiple objects sent in multipart message
- SMTP uses persistent connections
 - SMTP requires message (header & body) to be in 7-bit ASCII
 - SMTP server uses CRLF.CRLF to determine end of message

Mail message format

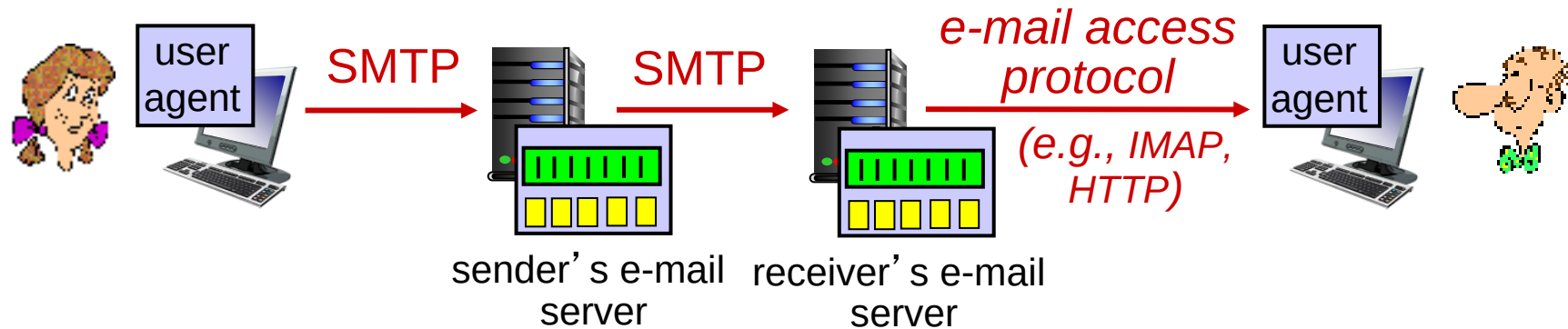
SMTP: protocol for exchanging e-mail messages, defined in RFC 5321 (like RFC 7231 defines HTTP)

RFC 2822 defines *syntax* for e-mail message itself (like HTML defines syntax for web documents)

- header lines, e.g.,
 - To:
 - From:
 - Subject:these lines, within the body of the email message area different from SMTP MAIL FROM:, RCPT TO: commands!
- Body: the “message” , ASCII characters only



Retrieving email: mail access protocols



- **SMTP:** delivery/storage of e-mail messages to receiver's server
- mail access protocol: retrieval from server
 - **IMAP:** Internet Mail Access Protocol [RFC 3501]: messages stored on server, IMAP provides retrieval, deletion, folders of stored messages on server
- **HTTP:** gmail, Hotmail, Yahoo!Mail, etc. provides web-based interface on top of SMTP (to send), IMAP (or POP) to retrieve e-mail messages

E-mail

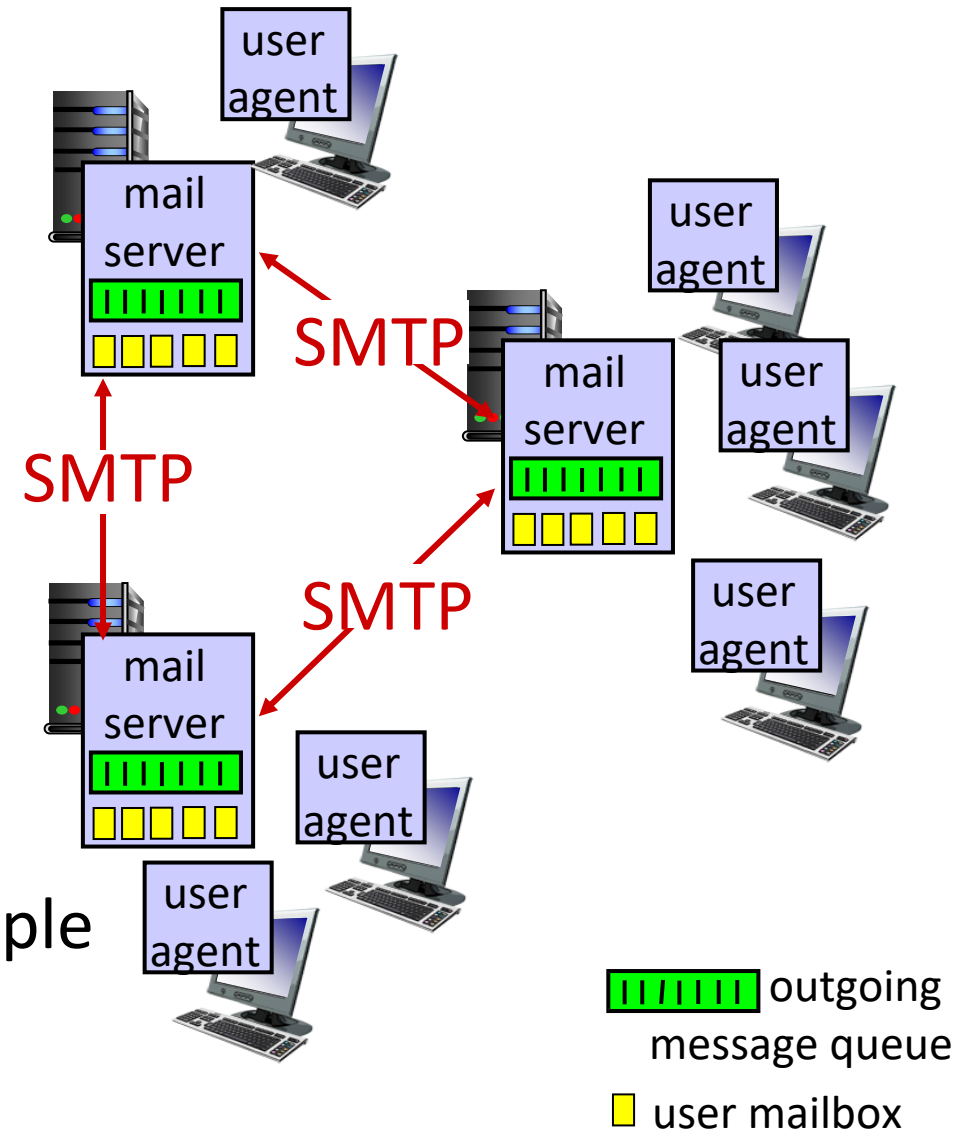
Three major components:

- user agents

- โปรแกรมที่ผู้ใช้ใช้ในการเขียนและส่งเมลล์ อ่านและรับเมลล์
- e.g., Outlook, iPhone mail client
- a.k.a. “mail reader”

- mail servers

- ถ้าเป็น คนส่ง mail server ฝั่งคนส่ง
- ถ้าเป็น คนรับ mail server ฝั่งคนรับ
- Email ปกติทั้งที่ส่ง และรับจะเก็บที่ mail server
- โปรโตคอลที่ mail servers ใช้ในการสื่อสาร (คุย) กัน simple mail transfer protocol: SMTP



Next : Video Streaming and
CDNs (to be continued)

Video Streaming and CDNs: context



Slide page



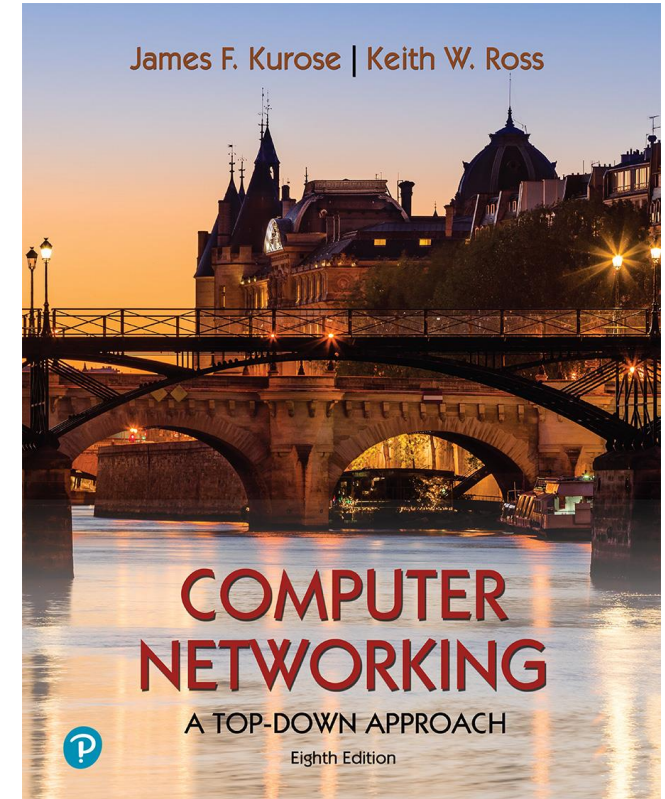
นำมาจาก **Powerpoint slides**
Chapter 2: Application Layer

Page 2-91 – 2-104 : Video Streaming and CDNs: context



Book : Computer Networking: A Top-Down Approach 8th edition
โดย **Jim Kurose, Keith Ross,**
Pearson, 2020

https://gaia.cs.umass.edu/kurose_ross/ppt.php



Computer Networking: A Top-Down Approach

8th edition

Jim Kurose, Keith Ross
Pearson, 2020

- How video is stored digitally?



Video Streaming and CDNs: context

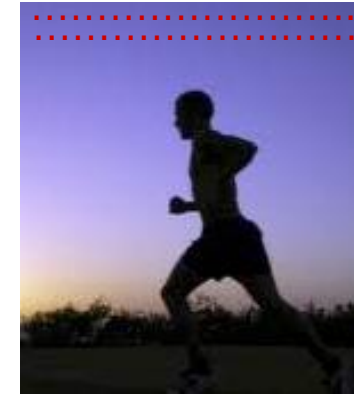
- stream video traffic: major consumer of Internet bandwidth
 - Netflix, YouTube, Amazon Prime: 80% of residential ISP traffic (2020)
- *challenge*: scale - how to reach ~1B users?
- *challenge*: heterogeneity
 - different users have different capabilities (e.g., wired versus mobile; bandwidth rich versus bandwidth poor)
- *solution*: distributed, application-level infrastructure



Multimedia: video

- digital image: array of pixels
 - each pixel represented by bits
- video: sequence of images displayed at constant rate
 - e.g., 24 images/sec
- coding: use redundancy *within* and *between* images to decrease # bits used to encode image
 - spatial (within image)
 - temporal (from one image to next)

spatial coding example: instead of sending N values of same color (all purple), send only two values: color value (*purple*) and number of repeated values (N)



frame i

temporal coding example: instead of sending complete frame at $i+1$, send only differences from frame i

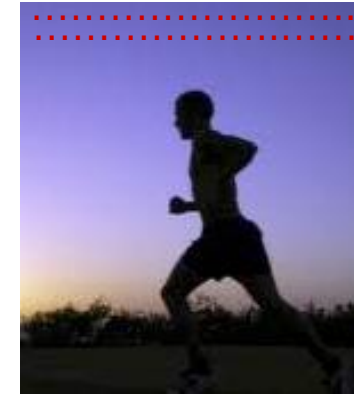


frame $i+1$

Multimedia: video

- **CBR: (constant bit rate):** video encoding rate fixed
- **VBR: (variable bit rate):** video encoding rate changes as amount of spatial, temporal coding changes
- **examples:**
 - MPEG 1 (CD-ROM) 1.5 Mbps
 - MPEG2 (DVD) 3-6 Mbps
 - MPEG4 (often used in Internet, 64Kbps – 12 Mbps)

spatial coding example: instead of sending N values of same color (all purple), send only two values: color value (*purple*) and number of repeated values (N)



frame i

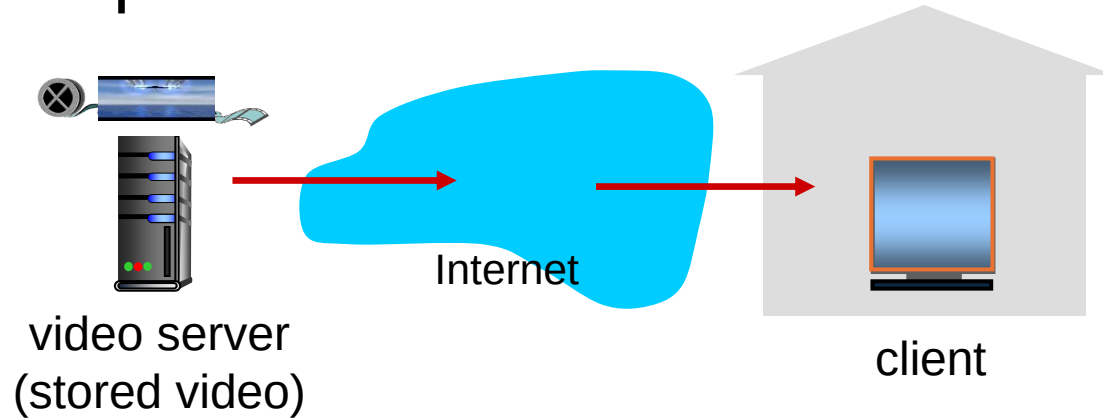
temporal coding example: instead of sending complete frame at $i+1$, send only differences from frame i



frame $i+1$

Streaming stored video

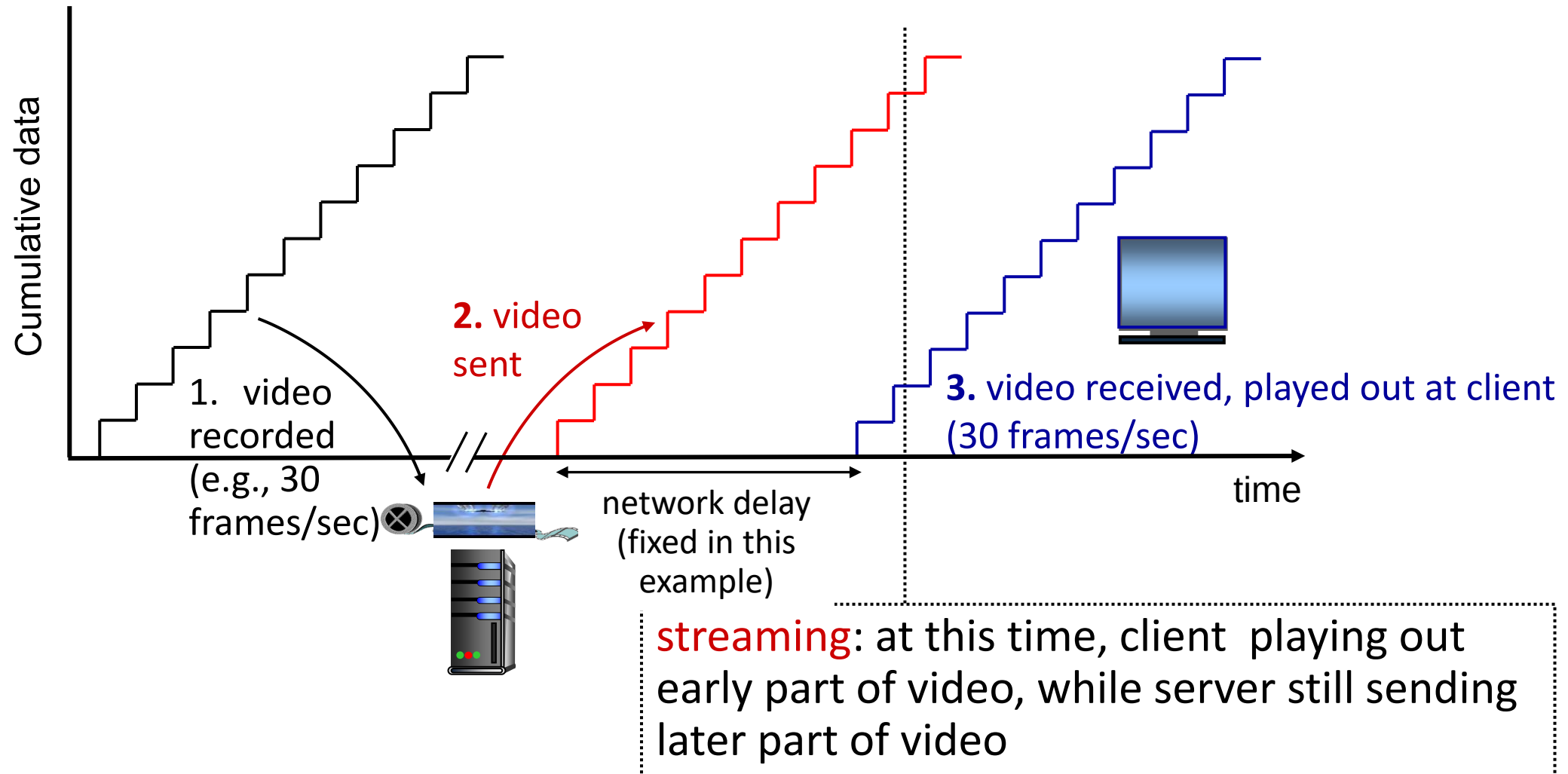
simple scenario:



Main challenges:

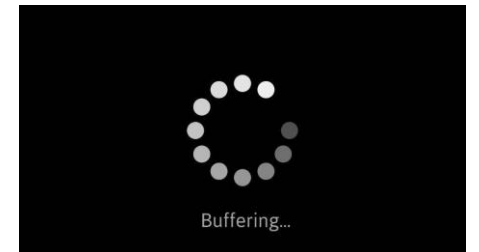
- server-to-client bandwidth will *vary* over time, with changing network congestion levels (in house, access network, network core, video server)
- packet loss, delay due to congestion will delay playout, or result in poor video quality

Streaming stored video

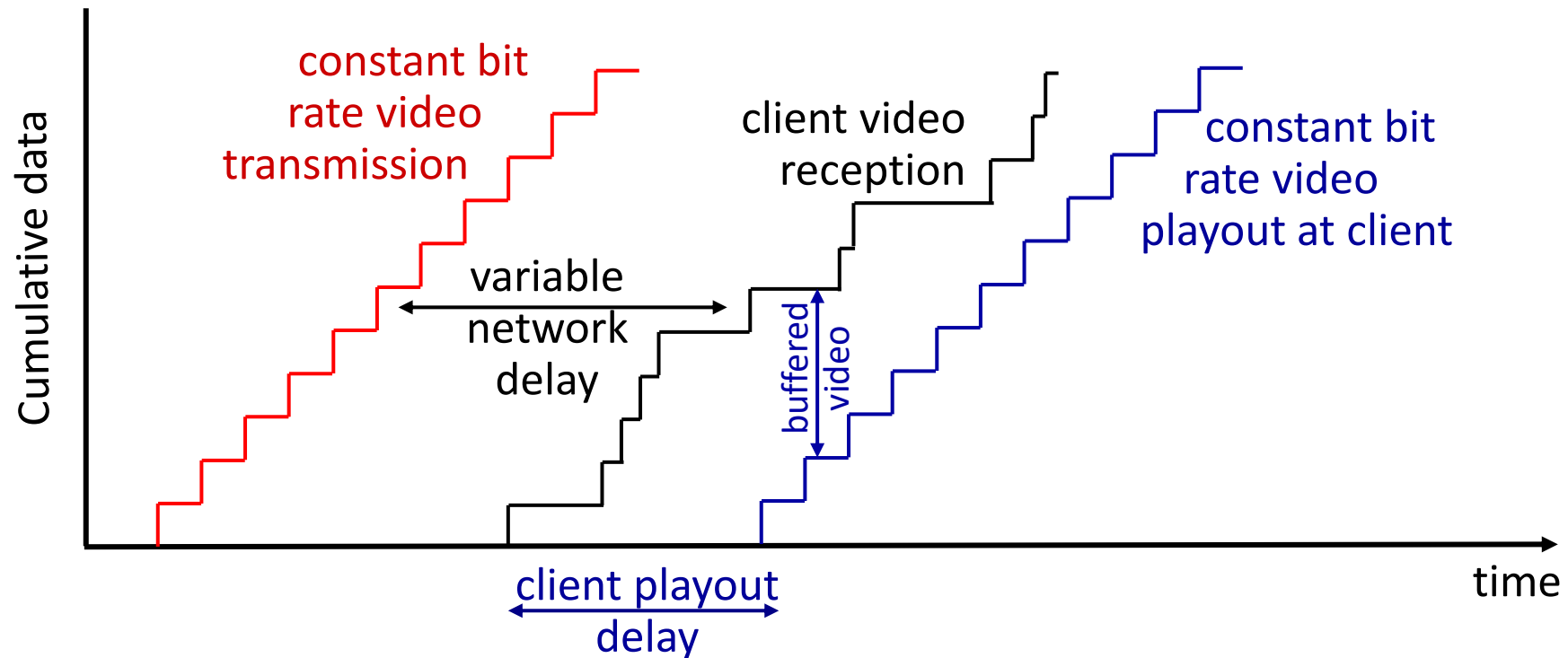


Streaming stored video: challenges

- **continuous playout constraint**: during client video playout, playout timing must match original timing
 - ... but **network delays are variable** (jitter), so will need **client-side buffer** to match continuous playout constraint
- other challenges:
 - client interactivity: pause, fast-forward, rewind, jump through video
 - video packets may be lost, retransmitted



Streaming stored video: playout buffering



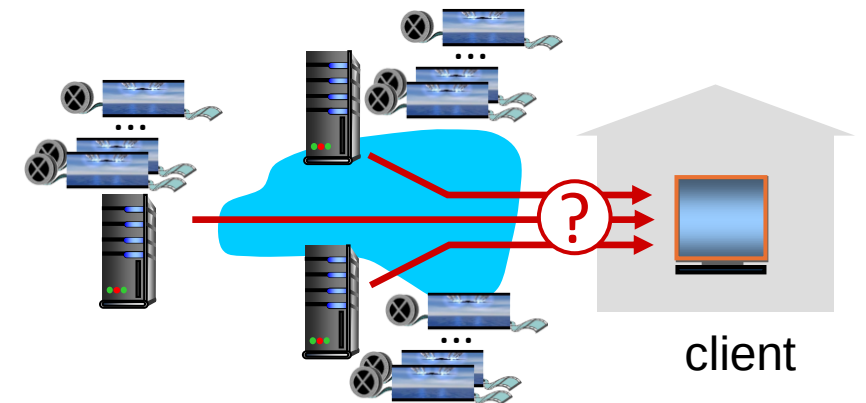
- *client-side buffering and playout delay*: compensate for network-added delay, delay jitter

Streaming multimedia: DASH

*D*ynamic, *A*daptive
*S*teaming over *H*TTP

server:

- divides video file into **multiple chunks**
- **each chunk** **encoded** at **multiple different rates**
- **different rate** encodings stored in **different files**
- files **replicated** in various CDN nodes
- *manifest file*: provides URLs for different chunks

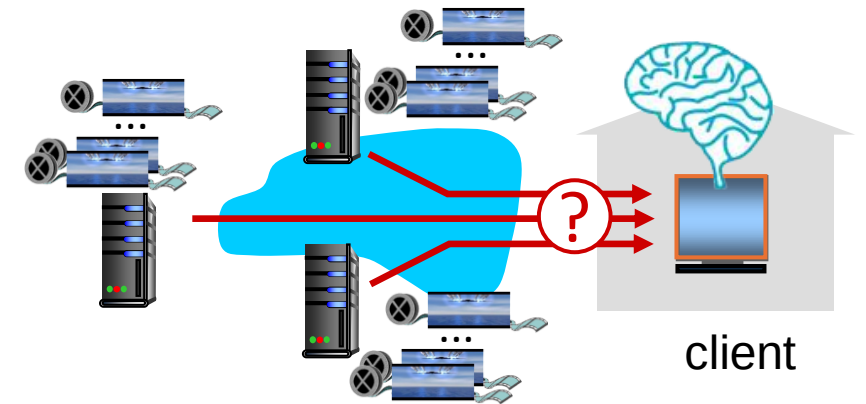


client:

- periodically estimates server-to-client bandwidth
- consulting manifest, requests one chunk at a time
 - chooses maximum coding rate sustainable given current bandwidth
 - can choose different coding rates at different points in time (depending on available bandwidth at time), and from different servers

Streaming multimedia: DASH

- “*intelligence*” at client: client determines
 - *when* to request chunk (so that buffer starvation, or overflow does not occur)
 - *what encoding rate* to request (higher quality when more bandwidth available)
 - *where* to request chunk (can request from URL server that is “close” to client or has high available bandwidth)

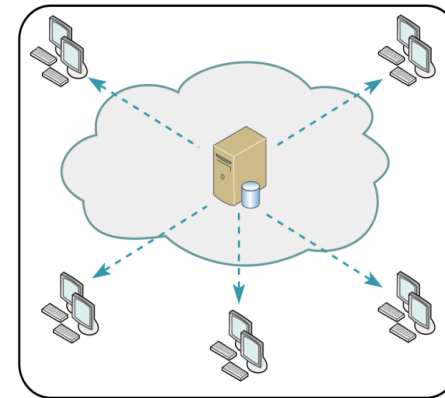


Streaming video = encoding + DASH + playout buffering

Content distribution networks (CDNs)

challenge: how to stream content (selected from millions of videos) to hundreds of thousands of *simultaneous* users?

- *option 1:* single, large “mega-server”
 - single point of failure
 - point of network congestion
 - long (and possibly congested) path to distant clients

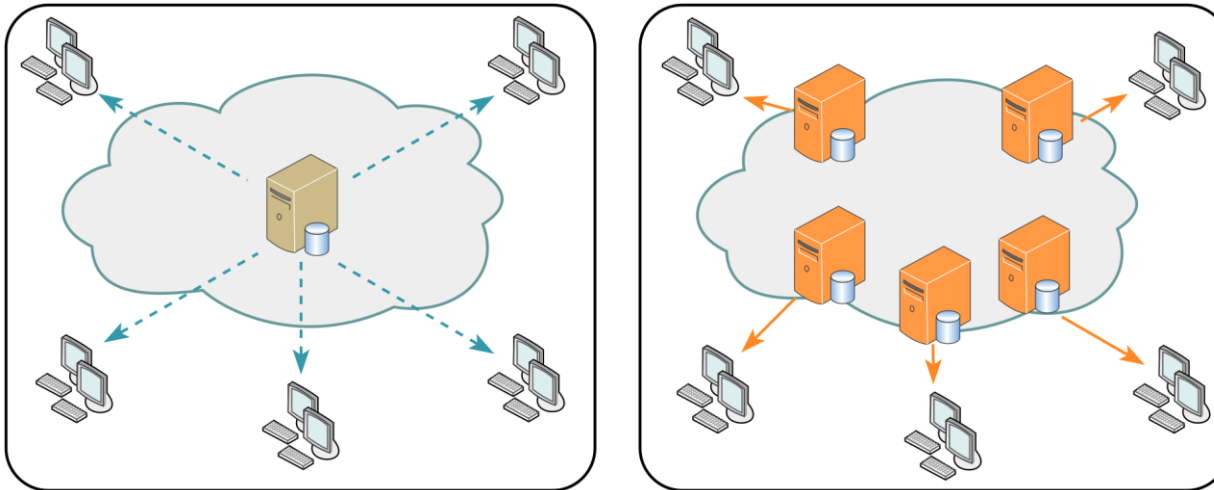


....quite simply: this solution *doesn't scale*

Content distribution networks (CDNs)

challenge: how to stream content (selected from millions of videos) to hundreds of thousands of *simultaneous* users?

- *option 2:* store/serve multiple copies of videos at multiple geographically distributed sites (*CDN*)



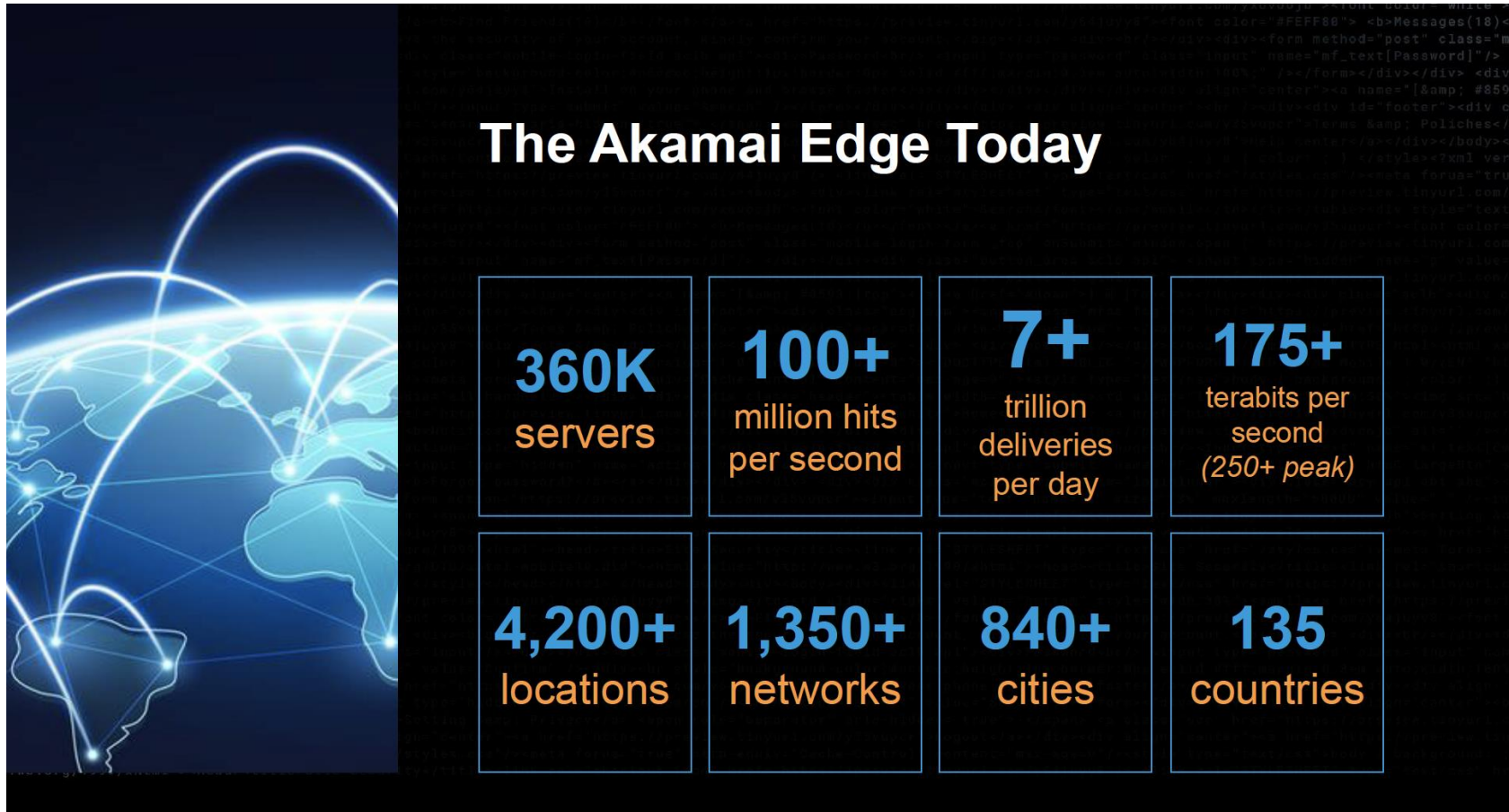
Content distribution networks (CDNs)

challenge: how to stream content (selected from millions of videos) to hundreds of thousands of *simultaneous* users?

- *option 2:* store/serve multiple copies of videos at multiple geographically distributed sites (*CDN*)
 - *enter deep:* push CDN servers deep into many access networks
 - close to users
 - Akamai: 240,000 servers deployed in > 120 countries (2015)
 - *bring home:* smaller number (10's) of larger clusters in POPs near access nets
 - used by Limelight



Akamai today:



Source: <https://networkingchannel.eu/living-on-the-edge-for-a-quarter-century-an-akamai-retrospective-downloads/>

How does Netflix work?

- Netflix: stores copies of content (e.g., MADMEN) at its (worldwide) OpenConnect CDN nodes
- subscriber requests content, service provider returns manifest
 - using manifest, client retrieves content at highest supportable rate
 - may choose different rate or copy if network path congested

